VIETNAM ACADEMY OF SCIENCE AND TECHNOLOGY
UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

BASIC PROGRAMMING

Lecture 5: Array and String

Dr. NGUYEN Hoang Ha
Dr. NGUYEN Minh Huong

Lecture 5:

- Array:
  - Why and What?
  - Declaring arrays
  - Accessing Array Elements
  - Arrays and Addresses
  - Passing and returning arrays to/from functions
- String:
  - String-array
  - String library functions

Array

Why Array?

How to store a list of 10 students' marks and compute the average?

- Using Individual variables:
  - Declaring 10 individual variables
  - Computing from 10 individual variables

    → cumbersome, repetitive writing

Compute Average with Individual Variables

```c
#include <stdio.h>
int main(void){
    int count = 10; /* Number of values to be read */
    long sum = 0L; /* Sum of the numbers */
    int mark1, mark2, mark3, mark4, mark5, mark6, mark7, mark8, mark9, mark10;
    float average = 0.0f; /* Average of the numbers */

    printf("\nEnter the 10 numbers:\n"); /* Prompt for the input */
    scanf("%d", &mark1);
    scanf("%d", &mark2);
    ...
    scanf("%d", &mark10);

    sum = mark1 + mark2 + mark3 + mark4 + mark5 + mark6 + mark7 + mark8 + mark9 + mark10;
    average = (float)sum/count; /* Calculate the average
    */
    printf("\nAverage of the ten numbers entered is: %f\n", average);
    return 0;
}
```

What is an Array?

- Idea:
  - A data structure to store consecutive individual variables
  - manage as a collection

Number 1	Number 2	Number 3	Number 4	Number 5

- Definition of Array: a data structure storing a fixed-size sequential collection of elements of the same type.

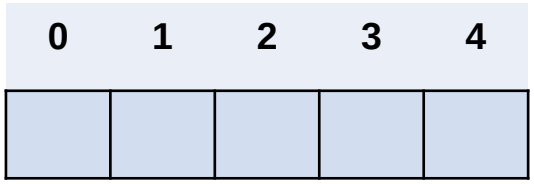
Declaring Arrays

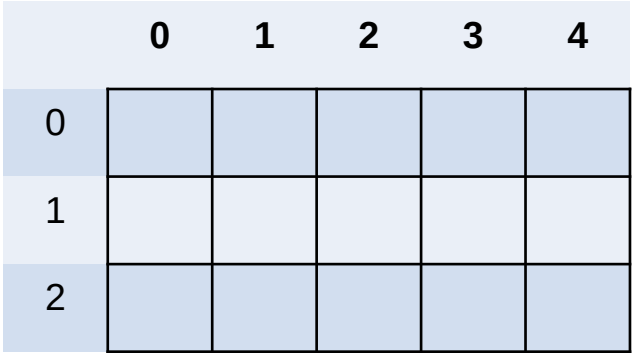
- Array declaration:
  - Data type of items
  - Variable name
  - Array size: number of items
- 1-D Arrays:

```
int A[5];
```

- 2-D Arrays:

```
int A[3][5];
```





Initializing Arrays

- Initializing with array size

```
int A[5] = {1, 2, 3, 4, 5};
```



Number of elements matches array size

- If array size is omitted, an array just big enough to hold the initialization is created

```
int A[] = {1, 2, 3, 4, 5};
```

Accessing Array Elements

- To access an element, you should point out:
  - Array name
  - The location of the element in the array

- One-dimensional arrays

```
// array declaration
int A[5] = {1, 2, 3, 4, 5};

for (int i =0; i<5; i++)
{
    printf("Element %d in the array is %d \n", i, A[i]);
}
```

0	1	2	3	4
1	2	3	4	5

Accessing Array Elements

- 2-dimensional arrays

```
// array declaration
int A[2][5] = {{1,2,3,4,5}, {6,7,8,9,10}};
for (int i =0; i<2; i++)
{
     for (int j =0; j<5; j++)
     {
               printf("%d \t", A[i][j]);
     }
     printf("\n");
}
```

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10

Accessing Array Elements

- 3-D arrays:

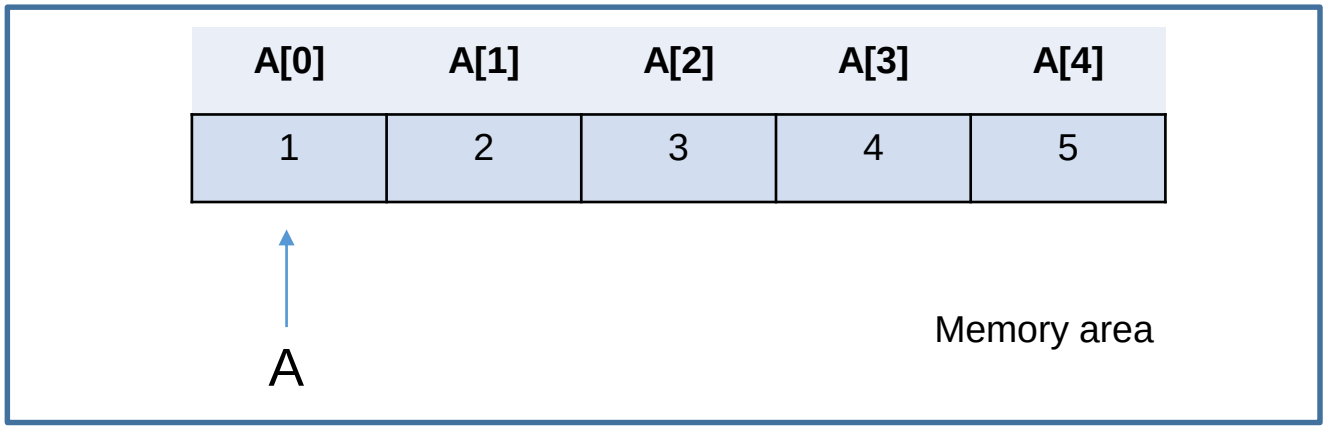
```
// array declaration
int A[2][2][3] = {
                    {
                        {1,2,3},
                        {4,5,6}
                    },
                    {
                        {7,8,9},
                        {10,11,12}
                    }
                };
```

```
for (int i =0; i<2; i++)
{
    printf("block %d \n",i);
    for (int j =0; j<2; j++)
    {
        for (int k = 0; k<3; k++)
        {
            printf("%d \t", A[i][j][k]);
        }
        printf("\n");
    }
}
```

Arrays and Address

- Array name is a **pointer** to the first element of the array (detailed in the next session)
- 1-D array:

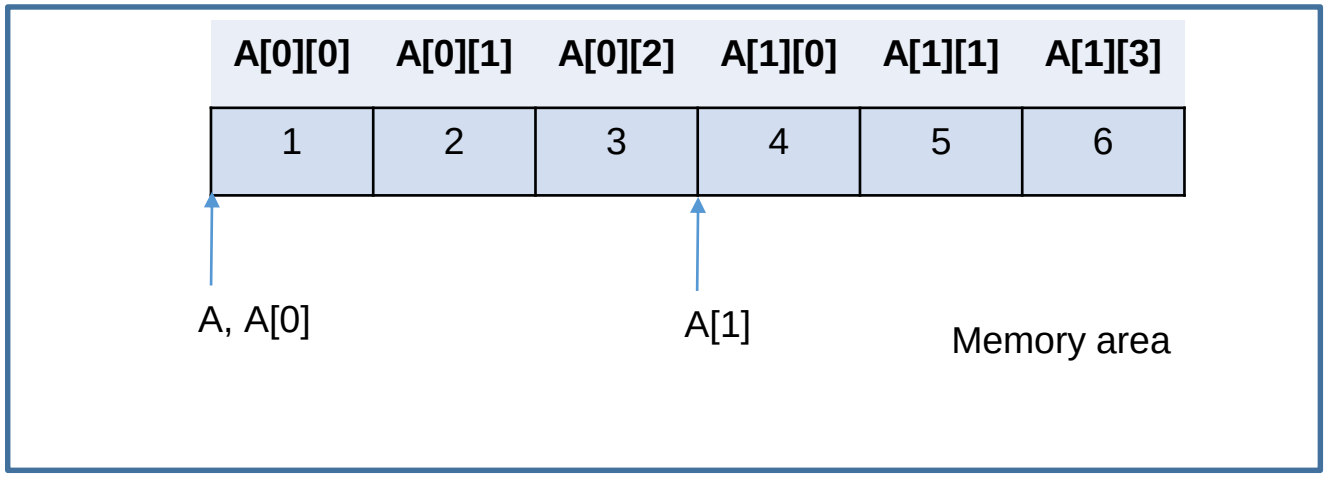
```
int A[5];
```



Arrays and Address

- 2-D array:

```
int A[2][3];
```



Arrays and Address

- Print out address of array elements

```
int A[2][3] = {
              {1,2,3},
              {4,5,6}
              };

for (int j =0; j<2; j++)
{
    for (int k = 0; k<3; k++)
    {
          printf("%p \t", &A[j][k]);
    }
    printf("\n");
}
```

Compute the Average using Array

```c
#include <stdio.h>
int main(void){
    int numbers[10]; /* Array storing 10 values */
    int count = 10; /* Number of values to be read */
    long sum = 0L; /* Sum of the numbers */
    float average = 0.0f; /* Average of the numbers */
    printf("\nEnter the 10 numbers:\n"); /* Prompt for the input */

    /* Read the ten numbers to be averaged */
    for(int i = 0; i < count; i++)
    {
        printf("%2d> ",i+1);
        scanf("%d", &numbers[i]); /* Read a number */
        sum += numbers[i]; /* Add it to sum */
    }

    average = (float)sum/count; /* Calculate the average*/
    printf("\nAverage of the ten numbers entered is: %f\n", average);
    return 0;
}
```

Passing Arrays to Functions

- As an unknown-size array
  - 1-D array: `void myfunction(int a[], int size)`
  - 2-D array: `void myfunction(int a[][3], int row)`
- As a fixed size array
  - 1-D array: `void myfunction(int a[5])`
  - 2-D array: `void myfunction(int a[5][3])`
- As a pointer
  - 1-D array: `void myfunction(int *a)`
  - 2-D array: `void myfunction(int **a)`

Passing Arrays to Functions

Compute_average.h

```
float compute_average (int numbers[], int size)
{
    float sum = 0L; /* Sum of the numbers */
    float avg = 0.0f; /* Average of the numbers */
    for(int i = 0; i < size; i ++)
        sum += numbers[i];
    if(size >0)
        avg = (float)sum/size;
    else
        avg = -1;//can't compute the average
    return avg;
}
```

Main.c

```
#include <stdio.h>
#include "compute_average.h"
int main(void){
    int numbers[10]; /* Array storing 10 values */
    int count = 10; /* Number of values to be read */
    long sum = 0L; /* Sum of the numbers */
    float average = 0.0f; /* Average of the numbers */
    printf("\nEnter the 10 numbers:\n"); /* Prompt for the input */

    /* Read the ten numbers to be averaged */
    for(int i = 0; i < count; i ++)
    {
        printf("%2d> ",i+1);
        scanf("%d", &numbers[i]); /* Read a number */
    }
    average = compute_average (numbers, count);
    printf("\nAverage of these numbers entered is: %f\n", average);
    return 0;
}
```

Returning Arrays from Functions

```
int* increase_elements(int arr[], int size)
{
    for (int i =0;i<size;i++)
    {
        arr[i] = arr[i] + 1;
    }
    return arr;
}
```

String

String Concept

- A string:
  - Sequence of characters
  - Is stored in an array of type char





Eg: String as an Array char

```c
#include <stdio.h>
int main(void){
    char gretting[] = "Hello";//compiler will automatically
        create sufficient space fo hold all characters declared*/

    int size = 5;
    for (int i =0;i<size;i++)
    {
        printf("The charactor #%d is %c\n", i+1, gretting[i]);
    }
    return 0;
}
```

Array of strings

```c
#include <stdio.h>
int main(void){
    char subject_names[5][30] = {"Linear Algebra", "Calculus", "Physics", "Co
mputer Architecture", "Basic Programming"};

    int size = 5;
    for (int i =0;i<size;i++)
    {
        printf("The subject #%d is %s\n", i+1, subject_names[i]);
    }
    return 0;
}
```

Standard Functions for Strings

- Declared in “string.h”
- Common functions

Function	Uses
strlen()	computes string's length
strcpy()	copies a string to another
strcat()	Concatenates (joins) two strings
strcmp()	compares two strings
strlwr()	converts string to lowercase
strupr()	converts string to uppercase

Practical time

- Create an 2d array to store the score of Calculus and Informatics scores of 10 students. Then calculate the average scores of each subject.
- Create and print out an 2D identity matrix of size n, n is inputted by the user.

1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1